VetRed — Red de Clínicas y Derivación de Laboratorio

Oct 23, 2026 · @Anto Dalmasso Fusco

1. Introducción

VetRed integra la atención clínica de una red de tres clínicas veterinarias con la derivación de muestras a un laboratorio central, y registra cada paso de la cadena de custodia desde la extracción hasta el diagnóstico.

El núcleo del modelo es la **muestra biológica**: su ciclo de vida (Extraída → Empaquetada → En tránsito → Recibida → Analizada, o Muestra Rechazada) se modela con el patrón State y cada cambio queda en una bitácora inmutable.

Aspecto	Decisión
Lenguaje	Python 3.11
Interfaz	Web local: Flask + plantillas Jinja + Bootstrap 5 (gráficos con Chart.js)
Persistencia	SQLite, acceso con SQL propio (sin ORM)
Dependencias externas	Solo Flask; Bootstrap y Chart.js van incluidos en el proyecto (funciona sin internet)
Alcance	7 procesos obligatorios, 3 opcionales, 5 patrones, 6 reportes
Pruebas	17 pruebas: 14 de negocio y 3 de la interfaz web

2. Modelo de dominio

El dominio tiene 23 entidades agrupadas en cuatro áreas; cada una valida sus invariantes al construirse (`dominio/entidades.py`), así un objeto inconsistente nunca llega a la base.

Área	Entidad	Atributos principales	Relación clave
Red	Clínica	nombre, dirección, teléfono	tiene veterinarios y conservadoras

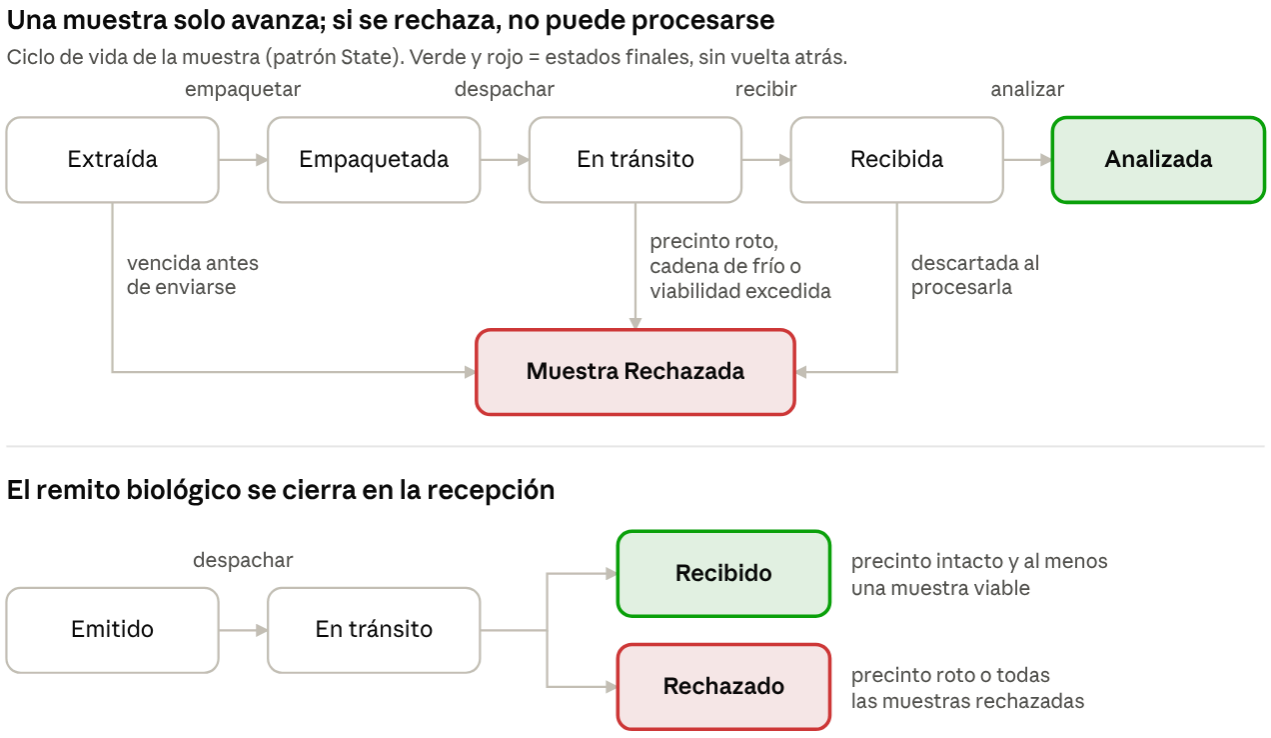
Área	Entidad	Atributos principales	Relación clave
Red	Laboratorio	nombre, dirección	destino de todos los remitos
Red	Veterinario	matrícula, especialidad, activo	pertenece a una clínica
Clientes	Cliente	DNI (único), nombre, contacto, activo	responsable de 1..n mascotas
Clientes	Mascota	especie, raza, sexo, nacimiento, peso	**siempre** tiene un cliente
Atención	Turno	inicio, fin, motivo, estado	veterinario + mascota
Atención	Consulta	fecha, tipo, motivo, diagnóstico, tratamiento, precio	forma la historia clínica
Atención	TipoVacuna / VacunaAplicada	especie, edad mínima, refuerzo (meses), lote	aplicada en una consulta
Atención	Comprobante / Ítem	número, total; tipo de servicio, subtotal	uno por consulta
Laboratorio	TipoAnálisis	tipo de muestra, rango °C, horas de viabilidad, precio	define cómo conservar la muestra
Laboratorio	OrdenDerivación	fecha, prioridad, observaciones	emitida desde una consulta
Laboratorio	Muestra	código, estado, motivo de rechazo	una por análisis de la orden
Laboratorio	HistorialEstado	estado anterior/nuevo, fecha, detalle	bitácora inmutable de la muestra
Custodia	Contenedor (conservadora)	código, rango °C, capacidad, estado	reutilizable, de una clínica
Custodia	Remito biológico	número, responsable, estado	agrupa muestras en un contenedor
Custodia	Precinto sanitario	código, estado, fecha de verificación	uno por remito
Custodia	Envío / EventoEnvío	transportista; ubicación, temperatura, hora	eslabones de la cadena

Área	Entidad	Atributos principales	Relación clave
Custodia	Recepción	recibido por, estado del precinto, °C llegada	cierra el remito
Custodia	Resultado	profesional, valores, conclusión	uno por muestra analizada
Avisos	Notificación	tipo, destinatario, mensaje, enviada	al cliente responsable

Regla de diseño importante: la muestra conoce su remito, y el remito conoce su contenedor y su precinto. Así, desde el código de una muestra se reconstruye toda su cadena de custodia.

3. Ciclo de vida de la muestra y del remito biológico

Una muestra recorre cinco estados y puede terminar en Muestra Rechazada desde la clínica, el transporte o el laboratorio; ningún estado final admite cambios.



estados de la muestra y del remito biológico

Cada flecha es un método de la clase de estado correspondiente; cualquier otra operación lanza `TransicionInvalidaError`. Cada transición queda registrada en la bitácora con fecha, estado anterior, estado nuevo y detalle (por ejemplo, el motivo del rechazo).

En la recepción, cada muestra se evalúa en este orden y se rechaza ante la primera falla:

1. Precinto sanitario roto: se rechazan todas las muestras del remito.
2. Alguna temperatura de la cadena de envío (salida, controles o llegada) fuera del rango del análisis.
3. Horas desde la extracción mayores que la viabilidad del análisis.

El remito usa una tabla de transiciones permitidas (`Remito.TRANSICIONES`). Al recibirse, la conservadora vuelve a quedar Disponible.

4. Procesos de negocio

Los 7 procesos obligatorios están implementados, cada uno con su pantalla y su servicio; además se cubren 3 de los opcionales.

#	Proceso	Pantalla	Servicio / método	Qué hace
1	Administrar clientes	Clientes	`ClienteService`	Alta, modificación, baja lógica y búsqueda
2	Mascotas e historias clínicas	Mascotas	`MascotaService.historia_clinica`	ABM; la historia reúne atenciones, vacunas, análisis y esquema de vacunación
3	Consultas y órdenes de derivación	Atención clínica	`ConsultaService.registrar`, `emitir_orden`	Registra la atención y las vacunas; la orden genera una muestra Extraída por análisis
4	Empaquetado	Logística › Empaquetado	`LogisticaService.empaquetar`	Coloca muestras en una conservadora y emite remito biológico y precinto sanitario
5	Cadena de envío	Logística › Cadena de envío	`despachar`, `registrar_punto_control`	Registra salida, controles en ruta con temperatura y responsable
6	Recepción en laboratorio	Logística › Recepción	`LogisticaService.recibir`	Verifica precinto, temperaturas y viabilidad; acepta o rechaza cada muestra

#	Proceso	Pantalla	Servicio / método	Qué hace
7	Resultados y comprobantes	Laboratorio, Comprobantes	`LaboratorioService`, `ComprobanteService`	Carga resultados; emite el comprobante con consulta, vacunas y análisis realizados

Opcionales implementados

- Agenda de turnos sin superposición (pestaña Agenda).
- Recordatorios de vacunas y avisos de resultados listos o muestras rechazadas (pestaña Avisos).
- Control de disponibilidad de conservadoras: quedan En uso mientras viajan y vuelven a Disponible al recibirse.

5. Validaciones de negocio

Las cinco validaciones de la consigna se aplican en dos niveles: en el dominio/servicio (mensaje claro al usuario) y, cuando es posible, también en la base de datos como última defensa.

Validación de la consigna	Dónde se aplica	Cómo
Toda mascota tiene un responsable	`Mascota.__post_init__`, `MascotaService`, columna `cliente_id NOT NULL`	Sin cliente activo no se crea la mascota; no se puede dar de baja un cliente con mascotas activas
Precinto roto impide el procesamiento	`LogisticaService.recibir`, estado `Rechazada`	Todas las muestras del remito pasan a Muestra Rechazada; `Rechazada.analizar()` lanza error
Trazabilidad inmutable de estados	Observer `RegistroTrazabilidad` + triggers SQL	Cada cambio se inserta en la bitácora; los triggers abortan cualquier UPDATE o DELETE sobre ella
Sin superposición de turnos	`TurnoService.reservar`	Rechaza un turno que se solape con otro del mismo veterinario o de la misma mascota, y fuera de 8 a 20 h

Validación de la consigna	Dónde se aplica	Cómo
Esquema de vacunas obligatorias	`VacunaService` al registrar la atención	Valida especie, edad mínima y refuerzo prematuro; alerta las obligatorias pendientes o vencidas

Validaciones adicionales del dominio

- La conservadora debe garantizar el rango térmico del análisis y no superar su capacidad.
- La recepción rechaza muestras si alguna temperatura de la cadena salió de rango (ruptura de cadena de frío) o si se superó su ventana de viabilidad.
- Las fechas de la trazabilidad deben ser cronológicas: ningún evento puede ser anterior al anterior.
- Una muestra en estado final (Analizada o Rechazada) no puede cambiar; lo garantiza también un trigger.
- El comprobante solo se emite cuando no quedan muestras sin resultado, y no factura las rechazadas.
- DNI único de 7 u 8 dígitos y email con formato válido.

6. Patrones de diseño

Se aplicaron cinco patrones (la consigna exige dos); State y Observer sostienen directamente el núcleo de trazabilidad.

Patrón	Archivo	Problema que resuelve
State	`dominio/muestra.py`	Las operaciones válidas dependen del estado de la muestra
Observer	`dominio/muestra.py`, `servicios/observadores.py`	Varios interesados reaccionan a cada cambio de estado
Template Method	`reportes/base.py`	Todos los reportes siguen el mismo algoritmo con pasos distintos

Patrón	Archivo	Problema que resuelve
Singleton	`persistencia/conexion.py`	Una única conexión compartida para que las transacciones abarquen varios servicios
Repository	`persistencia/repositorios.py`	Separar el SQL de alta, baja y modificación de la lógica de negocio

State

Cada estado (`Extraida` , `Empaquetada` , `EnTransito` , `Recibida` , `Analizada` , `Rechazada`) es una subclase de `EstadoMuestra` que redefine solo las operaciones que admite. La `Muestra` delega en su estado actual: `muestra.analizar()` llama a `estado_actual.analizar(muestra)` . Una operación no permitida lanza `TransicionInvalidaError` . Así se eliminan los `if estado == ...` dispersos y la regla "no se procesa una muestra rechazada" vive en un solo lugar: `Rechazada.analizar()` .

Observer

`Muestra` mantiene una lista de `ObservadorMuestra` y los notifica en cada transición. `RegistroTrazabilidad` inserta el cambio en la bitácora (y verifica el orden cronológico) y `AvisoAlCliente` genera una notificación cuando la muestra queda Analizada o Rechazada. Agregar otro efecto (por ejemplo, un mail real) no requiere tocar la clase `Muestra` .

Template Method

`Reporte.generar()` fija el algoritmo: `validar_filtros` → `obtener_datos` → `procesar` → `resumir` → `datos_grafico` . Solo `obtener_datos` es abstracto; el resto son hooks con comportamiento por defecto que cada reporte redefine si lo necesita. La pantalla de reportes trata a los seis por igual.

Singleton

`BaseDatos.__new__` devuelve siempre la misma instancia. Su método `transaccion()` es anidable: solo la transacción más externa hace COMMIT o ROLLBACK, de modo que si falla cualquier paso de un empaquetado (remito, precinto, muestras, contenedor) no queda nada a medias.

Repository

Un `Repositorio` genérico mapea cada dataclass con su tabla (`obtener`, `listar`, `insertar`, `actualizar`). Los servicios trabajan con objetos de dominio y no escriben SQL de ABM; las consultas con joins para listados y reportes quedan en servicios y reportes.

7. Reportes

Se implementaron seis reportes no triviales (la consigna pide cuatro), que cubren los cinco sugeridos. Todos se filtran por rango de fechas, muestran un resumen, un gráfico de barras y se exportan a CSV.

Reporte	Qué muestra	Por qué no es trivial
Atenciones clínicas y derivaciones	Atenciones por clínica y tipo, cuántas derivaron, % de derivación, muestras y órdenes urgentes	Agregación con subconsultas correlacionadas y porcentajes
Muestras rechazadas por fallas en el transporte	Detalle de cada rechazo con causa clasificada (precinto, cadena de frío, viabilidad) y tasa por transportista	Fecha de rechazo tomada de la bitácora; clasificación y tasas calculadas
Tratamientos aplicados	Atenciones con diagnóstico, tratamiento y vacunas; dosis aplicadas por vacuna	Concatena vacunas por atención y resume dosis
Vacunas obligatorias pendientes	Mascotas con vacunas nunca aplicadas o con refuerzo vencido y días de atraso	Se calcula en Python con la lógica del esquema (edad, intervalos)
Facturación por clínica y servicio	Importes de consultas, vacunas y laboratorio por clínica, % del total y ticket promedio	Pivote de ítems por tipo de servicio
Tiempos de tránsito y procesamiento	Horas promedio de espera, tránsito, procesamiento y total por clínica y análisis	Pivotea la bitácora de estados con una CTE y calcula diferencias con `julianday`

8. Arquitectura, persistencia, pruebas y ejecución

La aplicación se organiza en capas: la interfaz solo llama a servicios, los servicios usan el dominio y los repositorios, y nada fuera de `persistencia` abre conexiones.

Capa	Paquete	Responsabilidad
Presentación	`veterinaria/web`	Vistas Flask (una por área) y plantillas Bootstrap; el decorador `@validar` muestra los errores de negocio como alertas y vuelve a mostrar el formulario con lo cargado
Servicios	`veterinaria/servicios`	Un servicio por proceso; transacciones y validaciones entre entidades
Dominio	`veterinaria/dominio`	Entidades con invariantes y ciclo de vida de la muestra
Reportes	`veterinaria/reportes`	Plantilla base y seis reportes
Persistencia	`veterinaria/persistencia`	Esquema SQL, conexión y repositorios

Persistencia. SQLite con 23 tablas, claves foráneas activas, restricciones `CHECK` sobre los estados y tres triggers: dos impiden modificar o borrar la bitácora de estados y uno impide cambiar una muestra en estado final.

Datos de prueba. `datos_prueba.py` simula 75 días de actividad usando los mismos servicios, por lo que respeta todas las validaciones: 15 clientes, 25 mascotas, unas 290 atenciones y 300 muestras. Incluye precintos rotos, rupturas de cadena de frío, envíos demorados y mascotas con vacunas atrasadas, y deja muestras en cada estado para la demostración.

Pruebas. `tests/test_negocio.py` tiene 14 pruebas: responsable obligatorio, DNI duplicado, superposición de turnos, horario, esquema de vacunas, recorrido completo con trazabilidad, rechazo por precinto roto, ruptura de cadena de frío, inmutabilidad de la bitácora, transición inválida, conservadora no apta, comprobante con muestras pendientes y generación de los seis reportes.

Ejecución (Python 3.11, sin dependencias externas):

```
pip install -r requirements.txt   # solo la primera vez (instala Flask)
python main.py                    # levanta el servidor y abre
http://127.0.0.1:5000
python main.py --datos-prueba     # regenera la base de demostración
python -m unittest -v             # corre las pruebas
```

Guion sugerido para la demostración

1. Inicio: mostrar los indicadores y el circuito de muestras.
2. Atención clínica: elegir un paciente (aparece la alerta de vacunas) y usar “Registrar y derivar a laboratorio”.
3. Empaquetado: elegir la muestra nueva y una conservadora; se emiten remito y precinto.
4. Remitos y envíos: despachar y registrar un punto de control con 12 °C.
5. Recepción (en el mismo remito): precinto intacto; la muestra se rechaza por cadena de frío.
6. Resultados: intentar cargar el resultado de esa muestra (el sistema lo impide) y ver su trazabilidad.
7. Reportes: rechazos por transportista y tiempos de tránsito.